

Raport z implementacji mechanizmu stanu OAuth 2.0 w Juice Shop

Filip Horst 311257 Laboratorium BAIM 25L

1 Problem

Atak przez podstawienie access_code utworzony dla innego klienta (wykorzystano skrypt client.py). Wykonanie polega na dodaniu parametru access_code do głównej strony Juice Shop, czyli przykładowo:

```
GET http://localhost:3000/#access\_token=ya29.a0AeXRPp4hp2XkR0-e08k-UHNFVBNLJX bWT7 M9-xdz52H70v97eaiXCnC612 g KM44kAG-R0KDSY9rNur921NPHV6hkMLU03Ayd3oo5sNCMVCSLp5A9DhXpMsw4CS1SC7SPnwNUB1C DAHrUU5UG9gE0DKd7S-11ZpJwKMhwaCgYKARMSARISFQHGx2MifnKJkKjuLFfade0zPb2ehg0175
```

zwraca dostęp do strony jako zalogowany użytkownik, co nie powinno mieć miejsca ponieważ użyty kod nie był wygenerowany dla aplikacji Juice Shop.

2 Rozwiązanie

Przy wprowadzaniu zmian proces logowania i zapytania były monitorowane przez BurpSuite.

2.1 Tworzenie i zapis stanu

Stan jest tworzony przez prosty generator oparty o liczbę losową i konwersję na Base-36, co skutkuje zamianą na ciąg zawierający również litery. Przyjęty został stan długości 8 znaków – ogólnie pomysł zaczerpnięty ze StackOverflow.

Stan jest następnie wstawiany w localStorage przeglądarki z kluczem oauth_state.

Ostatnim etapem jest dołączenie stanu na koniec zapytania do oauthProvider, czyli w tym przypadku googleapis.

Zmiany wprowadzone zostały w pliku **frontend\src\app\login\login.component.ts**

```

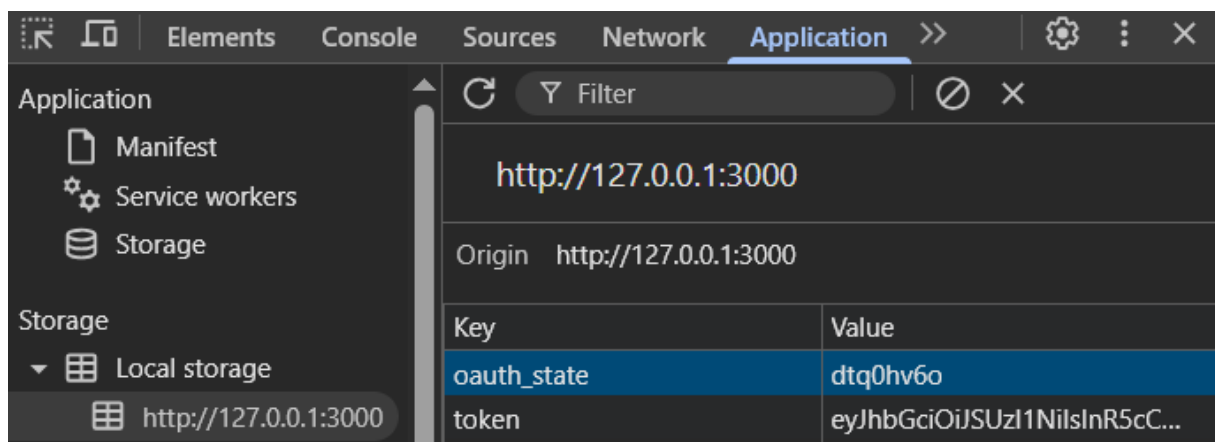
googleLogin () {
  //BAIM Generacja i zapis stanu OAuth
  const state = Math.random().toString(36).slice(2, 10);
  //https://stackoverflow.com/questions/1349404/generate-a-string-of-random-
  characters
  localStorage.setItem('oauth_state', state)
  console.log("BAIM 0204: Ustawianie stanu: " +
localStorage.getItem('oauth_state'))
  this.windowRefService.nativeWindow.location.replace(`${oauthProviderUrl}?c
lient_id=${this.clientId}&response_type=token&scope=email&redirect_uri=${this.
redirectUri}&state=${state}`)
}

```

Przykładowe zapytanie:

GET /o/oauth2/v2/auth?client_id=1005568560502-6hm161ef8oh46hr2d98vf2ohl nj4nfhq.apps.googleusercontent.com&response_type=token&scope=email&redirect_uri=https://local3000.owasp-juice.shop&state=dtq0hv6o

gdzie losowy stan wynosi dtq0hv6o. Jest on równocześnie zapisany w localStorage przeglądarki:



Zrzut konsoli DevTools po zalogowaniu, więc widać też token

2.2 Porównanie stanu zwracanego przez Authorization Server

```
ngOnInit () {  
    const params = this.parseRedirectUrlParams()  
    if (params.hasOwnProperty("state") && params.state ==  
localStorage.getItem('oauth_state')){  
        console.log("BAIM0204: Stan zgodny")  
        localStorage.removeItem('oauth_state')  
    } else {  
        console.log("BAIM0204: Stan niezgodny - odrzucenie")  
        localStorage.removeItem('oauth_state')  
        this.invalidateSession(new Error("Stan niezgodny"))  
        this.ngZone.run(async () => await this.router.navigate(['/login']))  
        return  
    }  
    this.userService.oauthLogin(this.parseRedirectUrlParams().access_token).su  
bscribe((profile: any) => {  
    [...]
```

Po wykryciu parametru access_code wykonywana jest funkcja ngOnInit w pliku **frontend\src\app\oauth\oauth.component.ts**. Jest to co prawda funkcja do rejestracji nowego użytkownika, ale jest uruchamiana przy każdym użyciu OAuth, więc już na tym etapie można przerwać atak. Porównanie jest banalnie proste i polega na przyrównaniu stanu zwracanego przez parametr URL (czyli ten zwracany przez googleapis przechodzący przez przekierowanie juiceshop3000). W sytuacji, gdy stan jest niezgodny sesja jest całkowicie odrzucana, co przerywa ten atak.

2.3 Inne zmiany

Serwis googleapis zwracał stan jako pierwszy parametr, natomiast aplikacja wyszukiwała tekstu „#access_token” w adresie, co skutkowało błędami. Naprawiono to przez usunięcie znaku „#” z wyszukiwania w pliku **frontend\src\app\app.routing.ts**

```
export function oauthMatcher (url: UrlSegment[]): UrlMatchResult {  
    [...]  
    const path = window.location.href  
    //Zmiana #access na access, bo googleapi zwraca state jako pierwszy parametr  
    if (path.includes('access_token=')) {  
        return ({ consumed: url })  
    }  
}
```

3 Testowanie

Warto zaznaczyć, że ze względu na sposoby czyszczenia pamięci strony stosowane w Juice Shop wykonywanie ataku i przebiegu poprawnego na przemian bez odświeżania strony może prowadzić do dziwnych rezultatów, ale po upewnieniu się, że nie pozostały żadne pozostałości po starym błędzie/logowaniu wszystko działało zgodnie z oczekiwaniami.

3.1 Przebieg poprawny (logowanie przez przycisk):

Pierwsze zapytanie do googleapis:

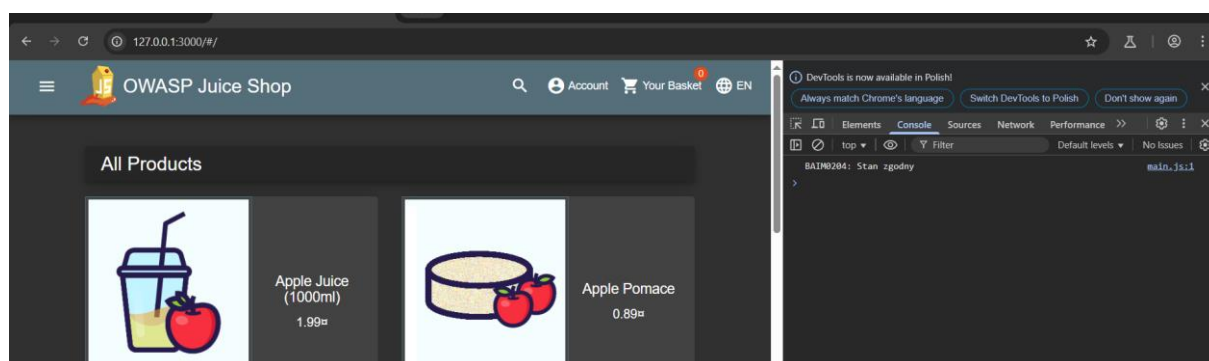
```
GET /o/oauth2/v2/auth?client_id=1005568560502-6hm161ef8oh46hr2d98vf2oh1nj4nfhq.apps.googleusercontent.com&response_type=token&scope=email&redirect_uri=https://local3000.owasp-juice.shop&state=i9o97fur
```

Generuje on przekierowanie do:

https://local3000.owasp-juice.shop#state=i9o97fur&access_token=ya29.a0AeXRPp5H7d9T0vxVdLuYD77UL9glOQldm3PtAnIQh5ngkUsysyJ0EyDNWcM6RX0VnqVXvyoOnqXumMDQdWuOktaPstGGYbwl1UNSTk8529BdCnMA2MaXpdxFHQyGTrGQi_NL_qusvLtyvC8g7hK2Fa3dQnXUYrt7xMJYTXzqaCgYKAakSARISFQHGX2Mi7LHD5KcBFmG0O_3c9tfulw0175&token_type=Bearer&expires_in=3599&scope=email%20openid%20https://www.googleapis.com/auth/userinfo.email&authuser=0&prompt=none

który przenosi użytkownika do strony lokalnego sklepu. Zarówno kod, jak i stan z założenia są tutaj poprawne, więc użytkownik jest wtedy zalogowany, czyli proces przebiegł poprawnie.

W konsoli można zaobserwować komunikat o zgodności stanu.



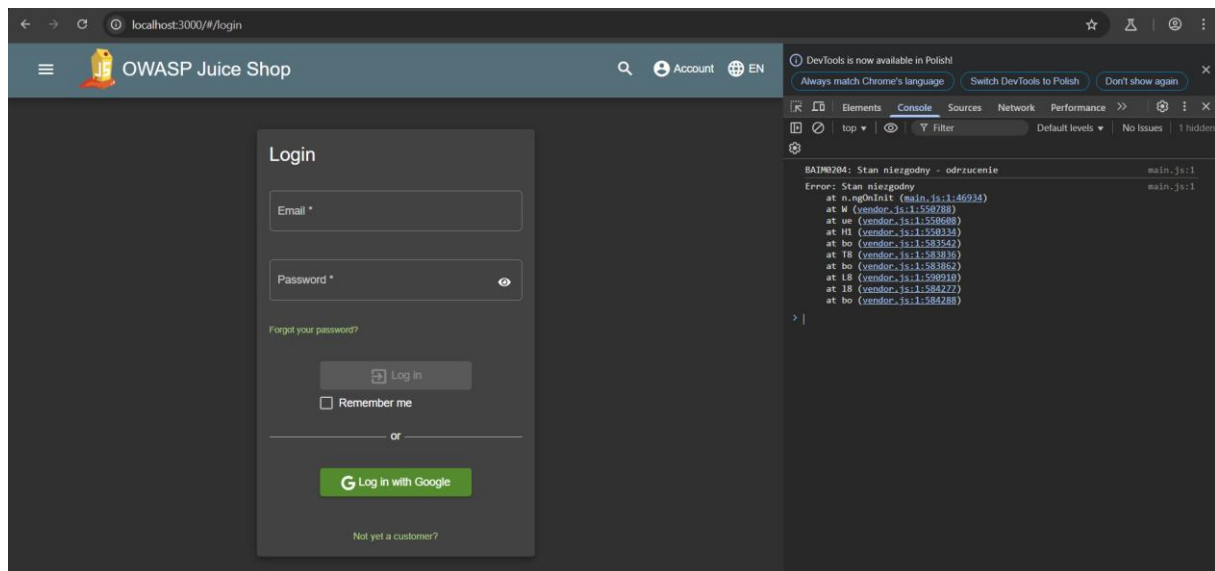
3.2 Wykonanie prostego ataku

Atakiem było przejście na adres

http://localhost:3000/#access_token=ya29.a0AeXRPp6rxMhRrEgFzQDv8cv6DotlZQOMaOPMfA35KzYdKMHSwDhmR37mXShminhcYTasTS5K_LQ8-JYuoLnd7YRn7ePCke9ygcigYJFjRQA8qrJBAzjWHHtEDigpHwF1PSEz7RqfsyaiSKZtV

[fs0ia5AzMTF1Ytg9WBGINMDaCgYKAX8SARISFQHGx2MiRT9JRJRbdvGFd012yj83Cw0175](http://localhost:3000/#access_token=ya29.a0AeXRPP6rxMhRrEgFzQDv8cv6DotlzQOMaOPMfA35KzYdKMHSwDhmR37mXShminhcYTasTS5K_LQ8-JYuoLnd7YRn7ePCke9ygcigYJfJrQA8qrJBazjWHHtEDigpHwF1PSEz7RqfsyaiSKZtVfs0ia5AzMTF1Ytg9WBGINMDaCgYKAX8SARISFQHGx2MiRT9JRJRbdvGFd012yj83Cw0175&state=4yahmg6v)

Dzięki wprowadzonym zmianom takie zapytanie skutkuje komunikatem o błędzie w konsoli (oczywiście w rzeczywistej wersji produkcyjnej nie była by udostępniana taka informacja) oraz przekierowaniem ponownie na stronę logowania:



Oznacza to, że zabezpieczenie zadziałało na atak przed podrzucenie kodu dostępu.

Warto jednak zaznaczyć, że jeśli w jakiś sposób dojdzie do sytuacji, gdzie atakujący uzyska dostęp do stanu znajdującego się w pamięci przeglądarki localStorage to atak wciąż można wykonać. Przykładowe zapytanie:

http://localhost:3000/#access_token=ya29.a0AeXRPP6rxMhRrEgFzQDv8cv6DotlzQOMaOPMfA35KzYdKMHSwDhmR37mXShminhcYTasTS5K_LQ8-JYuoLnd7YRn7ePCke9ygcigYJfJrQA8qrJBazjWHHtEDigpHwF1PSEz7RqfsyaiSKZtVfs0ia5AzMTF1Ytg9WBGINMDaCgYKAX8SARISFQHGx2MiRT9JRJRbdvGFd012yj83Cw0175&state=4yahmg6v